ITESO

Software Requirements Specification

Materia: Diseño de Sistemas Escalables

Profesor: David Emmanuel Ramírez Tovar

Alumnos: Federico Humberto Zaragoza Manuel

Ian Eugenio Rodriguez Wong

José Juan Diaz Campos

Sergio Isaac Santana Jiménez

Tabla de contenido

1. Introducción 3
1.1 Propósito 3
1.2 Alcance del proyecto 3
1.3 Definiciones, acrónimos y abreviaturas 3
1.4 Referencias 3
1.5 Visión general del documento 3
2. Descripción global 4
3. Requisitos específicos 4
3.1 Interfaces Externas 4
3.2 Requerimientos Funcionales 5
3.3 Requerimientos No Funcionales 8

1. Introducción

1.1 Propósito

Este documento define los requerimientos de software para la aplicación **MySpace**, una solución integral diseñada para modernizar y simplificar la administración de condominios residenciales. El sistema busca centralizar la comunicación, las finanzas y la gestión operativa en una sola plataforma, eliminando procesos manuales, dispersos y propensos a errores.

1.2 Alcance del proyecto

El sistema abarca una plataforma web dividida en varios módulos principales.

- **Módulo Administrativo:** Es el centro de mando para control total, dashboard financiero, gestión de propiedades, comunicación con los residentes y manejo de reservas de áreas comunes.
- **Módulo de Residente:** Este está enfocado a la autogestión, permitiendo pagos, reservas en línea de áreas comunes, gestión de visita (QR), comunicación con el administrador, reporte de fallas y una lista de movimientos recientes.
- **Arquitectura:** El sistema está pensado desarrollarlo sobre una infraestructura AWS Serverless utilizando un algoritmo a la medida para manejar picos de tráfico y priorización de usuarios.

1.3 Definiciones, acrónimos y abreviaturas

- AWS: Amazon Web Services
- QR: Quick Responsive Code
- RF: Requerimientos Funcionales
- RNF: Requerimientos no Funcionales

1.4 Referencias

- Guía IEEE 830-1998 - Recommended Practice for Software Requirements Specifications.

1.5 Visión general del documento

Este documento describe la funcionalidad global del sistema, seguido de los requerimientos de interfaces, funciones y atributos de calidad necesarios para el desarrollo y despliegue.

2. Descripción global

El sistema es una aplicación web responsiva y escalable. Se distingue por su capacidad de adaptarse a cargas de trabajo variables (picos de fin de mes o días festivos) mediante una arquitectura orientada a eventos.

El sistema interactúa con 2 tipos de usuarios:

1. Administradores: Que requieren herramientas de gestión masiva y financiera con alta prioridad de disponibilidad.
2. Residentes**:** Que requieren una interfaz sencilla para transacciones rápidas y reportes.

La lógica del sistema garantiza que la información de cada condominio esté aislada (seguridad y rendimiento) y que las operaciones críticas (pagos/administración) tengan preferencia sobre las operaciones estándar.

3. Requisitos específicos

3.1 Interfaces Externas

- **Interfaces de Usuario:**
  - Diseño responsivo adaptable a diferentes tamaños de pantalla (móvil, tablet, escritorio).
  - Interfaz limpia basada en tarjetas (cards) y ventanas modales para evitar recargas innecesarias.
  - Uso de paleta de colores semántica: Verde (OK/Al corriente), Rojo (Alertas/Deudas), Naranja (Pendientes).
- **Interfaces de Software:**
  - **Autenticación:** Integración con Amazon Cognito para gestión de sesiones y roles.
  - **Base de Datos:** Integración con Amazon DynamoDB particionada por Condominio_ID.
  - **Almacenamiento:** Integración con Amazon S3 para la carga de imágenes de reportes.
- **Interfaces de Comunicación:**
  - Protocolo HTTPS para toda la transmisión de datos.
  - API REST (Amazon API Gateway) para la comunicación entre el frontend y el backend.

3.2 Requerimientos Funcionales

ID	Módulo / Funcionalidad	Pre-condiciones	Happy Path (Flujo Principal)	Post-condiciones
RF-01	**Autenticación (Login general)**	El usuario debe existir en AWS Cognito.	1. Usuario ingresa credenciales. 2. Sistema valida en Cognito. 3. Se redirige al Dashboard según el rol (Admin/Residente).	Se almacena un Token JWT en el cliente. Sesión activa iniciada.
RF-02	**Gestión de Propiedades (Admin)**	Rol: Admin.	1. Selecciona "Agregar Propiedad". 2. Llena formulario dinámico (Casa/Depto). 3. Sistema guarda datos en DynamoDB.	Nueva propiedad listada con estado "OK".
RF-03	**Dashboard Financiero (Admin)**	Rol: Admin. Existen pagos previos.	1. Ingresa a "Finanzas". 2. Sistema calcula ingresos, gastos y cuentas por cobrar. 3. Renderiza gráficas y tabla.	Vista actualizada mostrada al usuario.

RF-04	**Exportación de Datos (Admin)**	Rol: Admin.	1. En vista financiera, clic en "Descargar". 2. Sistema genera archivo Excel con el estado de cuenta global.	Archivo .xlsx descargado en el dispositivo del Admin.
RF-05	**Revisión de Tickets (Admin)**	Rol: Admin. Existen tickets "Pendientes".	1. Abre ticket de la lista. 2. Revisa fotos y descripción. 3. Asigna proveedor del directorio. 4. Cambia estado a "En Proceso".	Ticket actualizado en BD. Residente ve el cambio en su portal.
RF-06	**Gestión de Reservas (Admin)**	Rol: Admin. Existen solicitudes.	1. Revisa reservas pendientes. 2. Selecciona "Aprobar". 3. Sistema bloquea fecha/hora en calendario general.	Reserva confirmada. Residente notificado.
RF-07	**Buzón de Avisos (Admin/Residente)** *(Reemplaza el Chat RF-19/27)*	Ambos roles.	1. Admin publica aviso general o Residente envía mensaje directo a la Admin.	Mensaje visible en la bandeja del receptor.

			2. Sistema almacena el mensaje.	
RF-08	**Pago de Servicios/Renta (Residente)**	Rol: Residente con saldo deudor.	1. Revisa estado de cuenta. 2. Selecciona deuda y sube foto del comprobante (Voucher). 3. Sistema envía a Admin para validación.	Deuda marcada como "Pago en Revisión". Imagen guardada en Amazon S3.
RF-09	**Generación de Tickets (Residente)**	Rol: Residente.	1. Selecciona "Nuevo Reporte". 2. Describe falla y adjunta foto. 3. Confirma envío.	Ticket en estado "Pendiente". Imagen en S3.
RF-10	**Reserva de Amenidades (Residente)**	Rol: Residente sin multas.	1. Abre calendario de áreas comunes. 2. Selecciona horario disponible. 3. Solicita reserva.	Reserva en estado "Pendiente de Aprobación".
RF-11	**Pases QR para Visitas (Residente)**	Rol: Residente.	1. Registra nombre de visita y fecha. 2. Solicita pase. 3. Sistema genera Token QR y lo muestra.	QR generado con fecha de caducidad.

RF-12	**Validación de Accesos (Guardia)** *(Nuevo Requerimiento)*	Rol: Guardia (o Admin).	1. Escanea QR del visitante. 2. Sistema valida vigencia del Token. 3. Muestra "Acceso Permitido".	Registro de "Entrada" guardado en el historial con Timestamp.

3.3 Requerimientos No Funcionales

3.3.1 Rendimiento (Performance)

- **RNF-01 (Latencia y Tiempos de Respuesta):** El 95% de las peticiones a la API REST (API Gateway) deben resolverse en menos de 300 milisegundos, garantizando transiciones fluidas en el *Frontend*. Se mitigarán los *Cold Starts* de AWS Lambda en las rutas críticas.

- **RNF-02 (Concurrencia Elástica):** El sistema debe soportar un escalamiento automático desde 0 hasta al menos 1,000 peticiones por segundo sin degradación del servicio, especialmente durante los días críticos de cobro (15 y 30 del mes).

- **RNF-03 (Desacoplamiento Asíncrono):** Operaciones computacionalmente pesadas o de larga duración (como la generación del Excel de Finanzas del RF-04) no deben bloquear la respuesta HTTP. Deben procesarse de forma asíncrona mediante el envío de un evento a Amazon SQS, notificando al usuario una vez concluido.

3.3.2 Seguridad y Privacidad

- **RNF-04 (Cifrado de Extremo a Extremo):** Toda la comunicación de red debe operar bajo el protocolo HTTPS (TLS 1.2+). Los datos almacenados en DynamoDB y S3 deben estar cifrados en reposo utilizando AES-256 (mediante AWS KMS).

- **RNF-05 (Autenticación sin Estado):** La sesión no debe depender de *cookies* de servidor. Se utilizarán Tokens JWT (JSON Web Tokens) gestionados por Amazon Cognito, con una expiración de 24 horas.

- **RNF-06 (Prevención de Vulnerabilidades):** Todas las entradas de datos en la API deben sanitizarse para evitar vulnerabilidades del OWASP Top 10 (ej. XSS y NoSQL Injection). Los datos sensibles (números de tarjetas, si aplican) no deben transitar por el backend de MySpace, delegándose a una pasarela PCI-DSS.

3.3.3 Escalabilidad y Arquitectura de Datos

- **RNF-07 (Aislamiento de Inquilinos - Multi-Tenancy):** La arquitectura de base de datos en DynamoDB utilizará el Condominio_ID como *Partition Key*. Esto garantiza el aislamiento lógico de los datos, previniendo que la saturación de lecturas/escrituras en un edificio masivo afecte el rendimiento de otro condominio.

- **RNF-08 (Escalamiento On-Demand):** Se prohíbe el aprovisionamiento de instancias fijas (EC2). Toda la infraestructura de cómputo y base de datos operará bajo un modelo *Pay-as-you-go / On-Demand*, reduciendo los costos operativos a cero durante periodos de inactividad nocturna.

3.3.4 Disponibilidad y Usabilidad

- **RNF-09 (Alta Disponibilidad Multi-AZ):** El sistema debe garantizar un *Uptime* (Acuerdo de Nivel de Servicio) del 99.9%. Todos los servicios gestionados (API Gateway, Lambda, DynamoDB) operarán nativamente replicados en múltiples Zonas de Disponibilidad (Multi-AZ) dentro de la misma región de AWS.

- **RNF-10 (Diseño Mobile-First y SPA):** La interfaz de usuario debe desarrollarse como una *Single Page Application* (SPA) responsiva. Debe cargar únicamente la estructura visual al inicio y consumir datos mediante llamadas asíncronas para evitar recargas completas de la página en los dispositivos móviles de los residentes.